Bài toán kiểm tra khả tuần tự

- Một lịch giao tác S sử dụng khóa nhị phân có đảm bảo khả tuần tự hay không?
- Giả sử:
  - S = {T1, T2, …, Tn}
  - Cần kiểm tra xem: Lịch S có **tương đương với một lịch tuần tự**?

Bài toán kiểm tra khả tuần tự

- Phương pháp:
  - Xây dựng đồ thị có hướng G = (N, E)
    - Mỗi giao tác là 1 đỉnh: N = {T1, T2, …, Tn}
    - Mỗi cạnh (có hướng):
      - Nếu Tj xin Lock(X) sau khi Ti Unlock(X) thì vẽ cạnh: Ti → Tj
  - Kiểm tra :
    - Đồ thị không có chu trình ➞ Lịch giao tác khả tuần tự
    - Đồ thị có chu trình ➞ Lịch giao tác không khả tuần tự

Bài toán kiểm tra khả tuần tự

- Ví dụ: Cho lịch giao tác S_1 gồm 2 giao tác T_1 và T_2 được biểu diễn như hình bên.
  - Kiểm tra S_1 có khả tuần tự không?



T_1	T_2
Lock(X)	
Read(X,t);	
t:=t+50;	
Write(X,t);	
UnLock(X)	
	Lock(X)
	Read(X,s);
	s:=s*2;
	Write(X,s);
	UnLock(X)
	Lock(Y)
	Read(Y,s);
	s:=s*2;
	Write(Y,s);
	UnLock(Y)
Lock(Y)	
Read(Y,t);	
t:=t+50;	
Write(Y,t)	
UnLock(Y)	

Hạn chế của khóa nhị phân

- Tại một thời điểm chỉ có một giao tác được truy cập dữ liệu X
  - Giảm hiệu suất hệ thống
  - Giảm khả năng thực hiện đồng thời
  - Nhiều giao tác phải chờ

Khóa đọc/ghi

- Ý tưởng:
  - Cho phép nhiều giao tác cùng đọc dữ liệu
  - Nhưng chỉ một giao tác được ghi dữ liệu
- ➔ Hai loại khóa:
  - Shared Lock (S) – Khóa chia sẻ
  - Exclusive Lock (X) – Khóa độc quyền

Khóa đọc/ghi

- Read Lock – Khóa đọc
  - Ký hiệu: Read_Lock(X)
  - Cho phép nhiều giao tác cùng đọc dữ liệu
- Write Lock – Khóa ghi
  - Ký hiệu: Write_Lock(X)
  - Chỉ một giao tác duy nhất được truy cập dữ liệu để ghi, đồng thời các giao tác khác không được đọc hoặc ghi trên dữ liệu này.
- UnLock – Giải phóng khóa
  - Ký hiệu: UnLock(X)

Khóa đọc/ghi

- Quy tắc:
  - Cấp khóa:
    - Đọc dữ liệu: Phải có Read_Lock(X) hoặc Write_Lock(X) trước khi Read(X).
    - Ghi dữ liệu: Phải có Write_Lock(X) trước khi Write(X).
  - Giải phóng khóa:
    - Chỉ Unlock(X) sau khi tất cả các thao tác đọc/ghi trên X đã hoàn tất.
    - Không được Unlock(X) nếu giao tác không thực sự nắm giữ khóa của X.
  - Chống trùng lắp: Nếu đang giữ bất kỳ khóa nào (Read/Write) trên X:
    - Không được xin cấp thêm Read_Lock(X) /Write_Lock(X).

Khóa đọc/ghi

- Nâng cấp / hạ cấp khóa:
  - Nâng cấp khóa: 1 giao tác T có thể xin nâng cấp khóa Read_Lock(X) → Write_Lock(X) khi T đang là giao tác duy nhất giữ khóa đọc
  - Hạ cấp khóa: 1 giao tác T có thể xin hạ cấp khóa Write_Lock(X) → Read_Lock(X) như vậy sẽ cho phép các giao tác khác đọc dữ liệu

Khóa đọc/ghi

- Ví dụ: Giao tác T1, T2 sử dụng khóa đọc/ghi

T_1
Read_Lock(Y)
Read(Y)
UnLock(Y)
Write_Lock(X)
Read(X)
X:=X+Y
Write(X)
UnLock(X)

T_2
Read_Lock(X)
Read(X)
UnLock(X)
Write_Lock(Y)
Read(Y)
Y:=Y+X
Write(Y)
UnLock(Y)

Khóa đọc/ghi

- Ví dụ: Giả sử 1 lịch S được lập từ T1, T2 và ban đầu X=20, Y=30 như hình bên
  - Kết quả X=50, Y=50
  - Nếu T1, T2 thực hiện tuần tự thì X=50, Y=80

  ➔S không khả tuần tự

T_1	T_2	X	Y
Read_Lock(Y)			
Read(Y)			30
UnLock(Y)			
	Read_Lock(X)		
	Read(X)	20	
	UnLock(X)		
	Write_Lock(Y)		
	Read(Y)		30
	Y:=Y+X		
	Write(Y)		50
	UnLock(Y)		
Write_Lock(X)			
Read(X)		20	
X:=X+Y			
Write(X)		50	
UnLock(X)			

Giao thức khóa 2 giai đoạn (Two Phase Locking – 2PL)

- Là một giao thức điều khiển tương tranh giúp:
  - Đảm bảo **lịch giao tác khả tuần tự (serializable)**
  - Hạn chế xung đột khi nhiều giao tác truy cập dữ liệu
- 1 giao tác phải thực hiện khóa theo 2 giai đoạn:
  - Giai đoạn 1: Xin cấp tất cả khóa
  - Giai đoạn 2: Giải phóng khóa

Giao thức khóa 2 giai đoạn (Two Phase Locking – 2PL)

- **Giai đoạn 1 – Growing Phase**: giai đoạn xin cấp khóa
  - Được phép xin khóa mới
  - Không được phép giải phóng khóa
  - ➔ số lượng khóa tăng dần → Growing Phase
  - Có thể xin nâng cấp khóa Read_Lock(X) → Write_Lock(X)
- **Giai đoạn 2 – Shrinking Phase**: giai đoạn giải phóng khóa
  - Được phép giải phóng khóa
  - Không được phép xin khóa mới
  - ➔ số lượng khóa giảm dần → Shrinking Phase
  - Có thể xin hạ cấp khóa Write_Lock(X) → Read_Lock(X)

Giao thức khóa 2 giai đoạn (Two Phase Locking – 2PL)

- Ví dụ: giao tác T sử dụng giao thức khóa 2 giao đoạn

```
Read_Lock(X)
Read(X)
Write_Lock(Y)
Write(Y)


Unlock(X)
Unlock(Y)
```

Giao thức khóa 2 giai đoạn (Two Phase Locking – 2PL)

- Ví dụ: 2 giao tác T1, T2 bên dưới có thực hiện giao thức khóa 2 giai đoạn không?

T_1	T_2
Lock(X)	Lock(Y)
Read(X)	Read(Y)
Lock(Y)	Lock(X)
Read(Y)	Read(X)
Y:=Y+X	UnLock(Y)
Write(Y)	X:=X+Y
UnLock(X)	Write(X)
UnLock(Y)	UnLock(X)

Giao thức khóa 2 giai đoạn (Two Phase Locking – 2PL)

- Ví dụ: 2 giao tác T3, T4 bên dưới có thực hiện giao thức khóa 2 giai đoạn không?

T_3	T_4
Lock(Y)	Lock(X)
Read(Y)	Read(X)
Y:=Y-100	UnLock(X)
Write(Y)	Lock(Y)
UnLock(Y)	Read(Y)
Lock(X)	UnLock(Y)
Read(X)	Print(X+Y)
X:=X+100	
Write(X)	
UnLock(X)	

Giao thức khóa 2 giai đoạn (Two Phase Locking – 2PL)

- Ví dụ: Chuyển 2 giao tác T1, T2 sau sang sử dụng giao thức khóa 2 giai đoạn

T_1	T_2
Read_Lock(Y)	Read_Lock(X)
Read(Y)	Read(X)
UnLock(Y)	UnLock(X)
Write_Lock(X)	Write_Lock(Y)
Read(X)	Read(Y)
X:=X+Y	Y:=Y+X
Write(X)	Write(Y)
UnLock(X)	UnLock(Y)



T_1,	T_2,
Read_Lock(Y)	Read_Lock(X)
Read(Y)	Read(X)
Write_Lock(X)	Write_Lock(Y)
UnLock(Y)	UnLock(X)
Read(X)	Read(Y)
X:=X+Y	Y:=Y+X
Write(X)	Write(Y)
UnLock(X)	UnLock(Y)

Giao thức khóa 2 giai đoạn (Two Phase Locking – 2PL)

- Ưu điểm:
  - Đảm bảo serializable schedule
  - Kiểm soát xung đột dữ liệu
  - Bảo vệ tính nhất quán của CSDL
- Nhược điểm:
  - Hạn chế số lượng giao tác truy cập đồng thời các đơn vị dữ liệu

Một số biến thể của 2PL

- Giao thức khóa hai giai đoạn bảo toàn
- Giao thức khóa hai giai đoạn chặt chẽ
- Giao thức khóa hai giai đoạn nghiêm ngặt

Giao thức nhãn thời gian

- Ý tưởng:
  - Mỗi giao tác sẽ được gán một nhãn thời gian
  - Hệ thống sẽ thực thi các giao tác theo thứ tự nhãn thời gian
- Mục tiêu:
  - Đảm bảo lịch giao tác khả tuần tự (serializable schedule)
  - Tránh deadlock vì không sử dụng khóa

Giao thức nhãn thời gian

- Nhãn thời gian (TimeStamps - TS)
  - Là định danh duy nhất do hệ quản trị CSDL gán cho mỗi giao tác.
  - Ký hiệu: TS(T)
  - Giá trị TS thường biểu thị cho thời điểm giao tác bắt đầu
  - Ví dụ: TS(T1) < TS(T2) → giao tác T1 phải thực hiện trước T2

Giao thức nhãn thời gian

- Phương pháp tạo timestamp:
  - Sử dụng bộ đếm
    - Dễ triển khai
    - Tuy nhiên bộ đếm có giới hạn nên cần reset khi hệ thống rãnh
  - Sử dụng thời gian hệ thống
    - Phải đảm bảo không có 2 giai tác có cùng timestamp

Thuật toán thứ tự nhãn thời gian

- Ý tưởng: sắp xếp các giao tác theo thứ tự timestamp.
- Để đảm bảo tránh xung đột, thuật toán cần phải gán 2 giá trị nhãn thời gian cho đơn vị X như sau:
  - Read_TS(X): Timestamp lớn nhất của giao tác đã đọc X.
    - Read_TS(X) = TS(T), trong đó T là giao sau cùng đọc đơn vị dữ liệu X thành công.
  - Write_TS(X): Timestamp lớn nhất của giao tác đã ghi X.
    - Write_TS (X) = TS(T), trong đó T là giao tác sau cùng ghi đơn vị dữ liệu X thành công.
- 2 loại thuật toán:
  - Thứ tự nhãn thời gian cơ bản (TO).
  - Thứ tự nhãn thời gian chặt (Strict TO).

Thứ tự nhãn thời gian cơ bản (TO)

- Khi giao tác T đọc dữ liệu X:
  - Nếu Write_TS(X) > TS(T): giao tác T bị rollback vì một giao tác "trẻ" hơn đã ghi X trước.
  - Ngược lại, cho phép đọc, cập nhật Read_TS(X) = TS(T)
- Khi giao tác T ghi dữ liệu X:
  - Nếu Read_TS(X) > TS(T) hoặc Write_TS(X) > TS(T): giao tác T bị rollback vì một giao tác "trẻ" hơn đã đọc/ghi X trước.
  - Ngược lại, cho phép ghi, cập nhật Write_TS(X) = TS(T)

Thứ tự nhãn thời gian cơ bản (TO)

- Hiện tượng Cascading Rollback
  - Khi một giao tác bị rollback, các giao tác khác phụ thuộc cũng phải rollback.
  - Ví dụ:
    - T1 ghi X, T2 đọc X, T3 đọc X
    - T1 rollback → T2, T3 rollback theo

Thứ tự nhãn thời gian chặt (Strict TO)

- Cải tiến từ thuật toán TO
- Giải quyết vấn đề Cascading Rollback
- Nguyên tắc hoạt động:
  - Nếu giao tác T muốn đọc hoặc ghi X nhưng: TS(T) > Write_TS(X) thì T phải chờ giao tác trước hoàn thành

  ➔ Kết quả: tránh Cascading Rollback, dễ phục hồi dữ liệu.